VECTOR

MICROSAR I2C

Technical Reference

DrvI2c_S32K14xFsli2c01Asr

Version 2.0.0

Document Information

History

Author	Date	Version	Remarks
virmid	2017-06-10	1.00.00	Initial version of this technical reference.
fbatz	2021-08-20	2.00.00	Upgrade driver to I2C core version 5.1.3.

Reference Documents

No.	Source	Title	Version
[1]	Vector	Technical Reference MICROSAR I2C	see delivery
[2]	Vector	Technical Reference MICROSAR OS	see delivery
[3]	NXP	S32K3xx Reference Manual	Rev.2 Draft C 04/2021



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2021 Vector Informatik GmbH

Contents

1 Introduction 5

2 Functional Description 6
2.1 Features 6
2.1.1 Limitations 6

3 Integration 7
3.1 Embedded Implementation 7
3.2 Critical Sections 7
3.3 Dependencies on SW modules 7
3.3.1 Dio 7
3.3.2 Port 8
3.3.3 OS 8

4 Extended API Description 9
4.1 Interrupt Service Routines provided by I2C 9
4.1.1 I2C interrupt service 9
4.2 Services used by I2C 9

5 Configuration 11
5.1 Configuration with DaVinci Configurator Pro 11
5.1.1 General configuration hints 11
5.1.2 I2C Bus configuration 11
5.1.3 I2C Hardware configuration 12
5.1.4 Configuration of the nine clock cycles 12
5.1.5 Base address 13

6 Glossary and Abbreviations 14
6.1 Glossary 14
6.2 Abbreviations 14

7 Contact 15

Illustrations

Figure 5-1 Configuration items in the I2cGeneral container 11
Figure 5-2 Bus configuration 11
Figure 5-3 Hardware configuration 12
Figure 5-4 Configuration of the Dio and Port module 12
Figure 5-5 Configuration of the port driver interface 12
Figure 5-6 Base address of the peripheral units 13

Tables

Table 3-1 Implementation Files 7
Table 3-2 Critical Sections 7
Table 4-1 I2c_IrqUnit[x] 9
Table 4-2 Services used by I2C 9
Table 6-1 Glossary 14
Table 6-2 Abbreviations 14

1 Introduction

This document is a supplement to the Technical Reference of MICROSAR I2C [1]. It describes specific peculiarities of Drvl2c_S32K14xFsli2c01Asr.

2 Functional Description

2.1 Features

All features are listed in the Technical Reference of MICROSAR I2C [1].

Only limitations of the Drvl2c_S32K14xFsli2c01Asr are described in the following chapter.

2.1.1 Limitations

The Drvl2c_S32K14xFsli2c01Asr can only receive a maximum number of 256 Bytes per transmission. This hardware limitation is described in [3] chapter 67.3.1.

Due to this hardware limitation, the part of the driver implementation that deals with handling more than 256 bytes has been adjusted. If the user triggers a RX transmission with a larger number of bytes, the driver will automatically split up the transmission in multiple parts by generating a repeated start condition.

3 Integration

This chapter gives necessary information for the integration of the MICROSAR I2C into an application environment of an ECU.

3.1 Embedded Implementation

The delivery of the I2C module contains these source code files:

File Name	Description
I2c_PerMgr.c	Source file I2c's hardware specific sub-module Peripheral Manager.
I2c_PerMgr.h	Header file of I2c's hardware specific sub-module Peripheral Manager.
I2c_PerMgr_RegAccess_Int.h	Includes functions to access hardware registers.
I2c_Regs.h	Includes all register addresses.
I2c_S32K14xFsli2c01_bswmd.arxml	Hardware specific description file.
DrvI2c_S32K14xFsli2c01Asr.jar	Hardware specific generator plugin for DaVinci Configurator 5.

Table 3-1 Implementation Files

3.2 Critical Sections

The DrvI2c_S32K14xFsli2c01Asr uses one critical section in two functions.

Critical Section Define	Description
I2C_EXCLUSIVE_AREA_0	Used inside I2c_PerMgr_RxData() to ensure consistency of received data. > Duration is short (< 20 instructions) > API call to OS module if Protected Mode is enabled, otherwise no API call to other BSW modules inside. > Disable global interrupts.
I2C_EXCLUSIVE_AREA_0	Used inside I2c_PerMgr_FreeBus() to ensure the generation of a clean, uniform clock pulse signal, which might be required by external slave devices. > Duration is short (< 30 instructions) > API call to Dio module. > Disable global interrupts.

Table 3-2 Critical Sections

3.3 Dependencies on SW modules

3.3.1 Dio

The Dio module is needed to generate nine clock pulses. The clock pulses try to free the bus when the SDA line is stuck low.

3.3.2 Port

The Port module is necessary to release the bus when the SDA line is pulled low by the slave. With the Port module the mode can be changed between I2C and GPIO mode.

3.3.3 OS

Depending on the configuration (parameter I2cProtectedMode), the memory-mapped registers are either accessed directly or via the APIs offered by OS.

4 Extended API Description

4.1 Interrupt Service Routines provided by I2C

The I2C provides an interrupt service routine for each hardware unit.

4.1.1 I2C interrupt service

Prototype	
`void` I2c_IrqUnit[x] `( void )`	
Parameter	
--	--
Return code	
--	--
Functional Description	
Interrupt service for TX and RX error handling of hardware unit [x]. The [x] is the hardware unit index. It is 0 for standard I2C support and > 0 as the chip supports more than one hardware unit	
Particularities and Limitations	
> This function is asynchronous. > This function is non-reentrant. > This function is called in interrupt context of the OS.	

Table 4-1 I2c_IrqUnit[x]



Caution
The interrupts for I2C are enabled by an AUTOSAR OS or must be enabled externally in the interrupt module before using this driver.

4.2 Services used by I2C

The following table shows services provided by other components, which are used by the I2C. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
OS	osReadPeripheral32 osWritePeripheral32
Dio	Dio_WriteChannel
Port	Port_SetPinMode

Table 4-2 Services used by I2C



Caution
In case no MICROSAR OS is present, the following must be considered when using the Protected Mode:

> Provide osReadPeripheral32 and osWritePeripheral32 functionality with the signature described in [2] Technical Reference MICROSAR OS.

© 2021 Vector Informatik GmbH

5 Configuration

In the I2C module the attributes can be configured according to/ with the following methods/ tools:

> Configuration in DaVinci Configurator (Base/Pro)

5.1 Configuration with DaVinci Configurator Pro

5.1.1 General configuration hints

The driver generates nine clock cycles to realize the “free bus” scenario. This is automatically triggered if the driver runs into a bus timeout. The macros are preset as default and can be changed.



Basic Knowledge

The macros for generating the nine clock cycles, which are used by the I2C module, can be configured. The mapping is:

```
#define I2C_DioWriteChannel(ChannelId,Level)    Dio_WriteChannel(ChannelId,Level)
#define I2C_PortSetPinMode(Pin,Mode)            Port_SetPinMode(Pin,Mode)
```

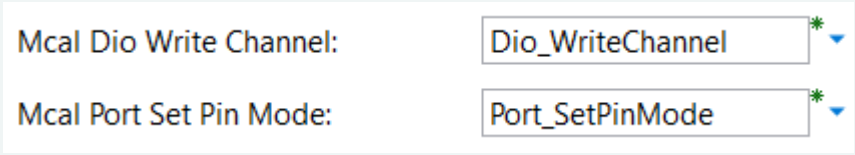


Figure 5-1 Configuration items in the I2cGeneral container

5.1.2 I2C Bus configuration

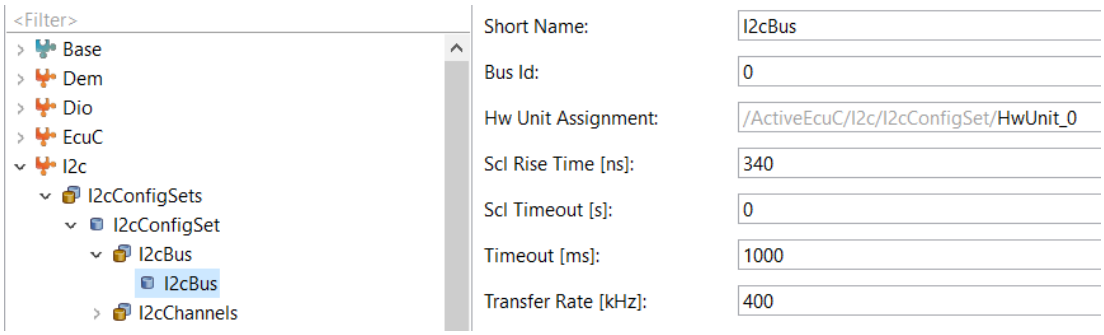


Figure 5-2 Bus configuration

A bus configuration is assigned to a sequence. All channels in a sequence inherit the properties of this bus configuration. The “Timeout” uses the I2C drivers main function cycle. If the I2C runs in a millisecond schedule and the timeout is set to 1000 ms, the generator will calculate the correct number of loops until the timeout is hit and the “free bus” is engaged.

© 2021 Vector Informatik GmbH

5.1.3 I2C Hardware configuration

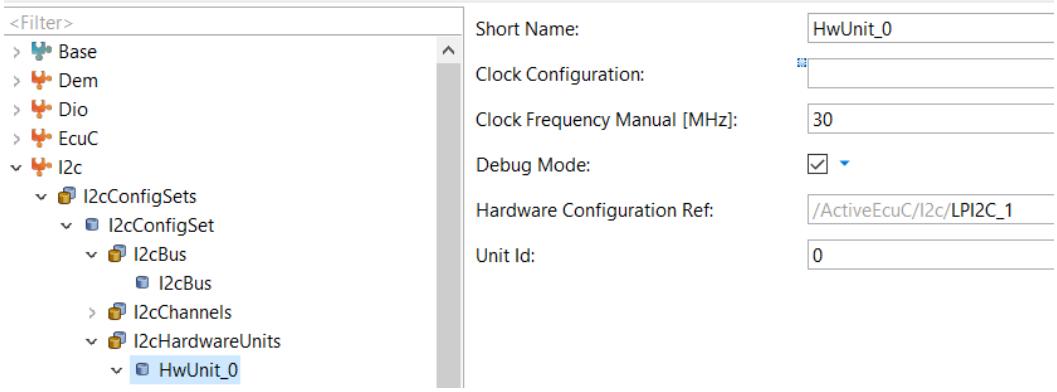


Figure 5-3 Hardware configuration

The hardware configuration is assigned to the module of this sequence and specifies the hardware properties. The clock configuration of the module can be set with the parameter “Clock Configuration” or “Clock Frequency Manual”. If the reference of the clock configuration is set, the Clock Frequency Manual can be ignored. This configuration is very important for the baudrate calculation.

5.1.4 Configuration of the nine clock cycles

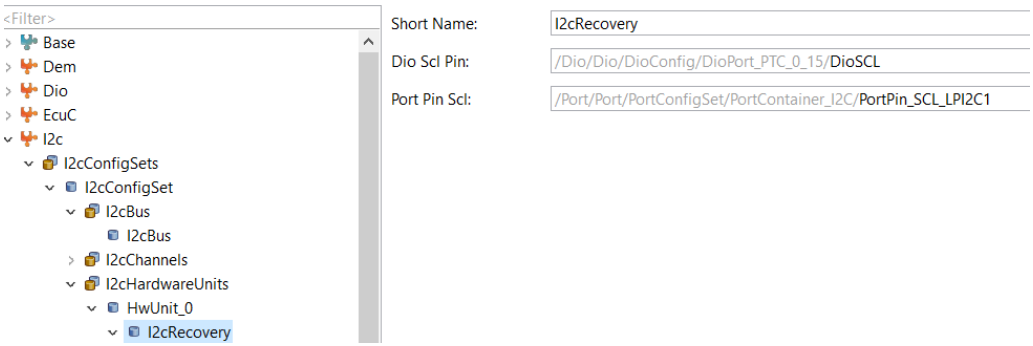


Figure 5-4 Configuration of the Dio and Port module

The driver must generate the nine clock cycles by itself. To ensure the correct behavior, the user must configure the references to the corresponding Dio and Port module. If the configuration was successful, the driver will use its SCL pin as GPIO to generate the clock cycles.

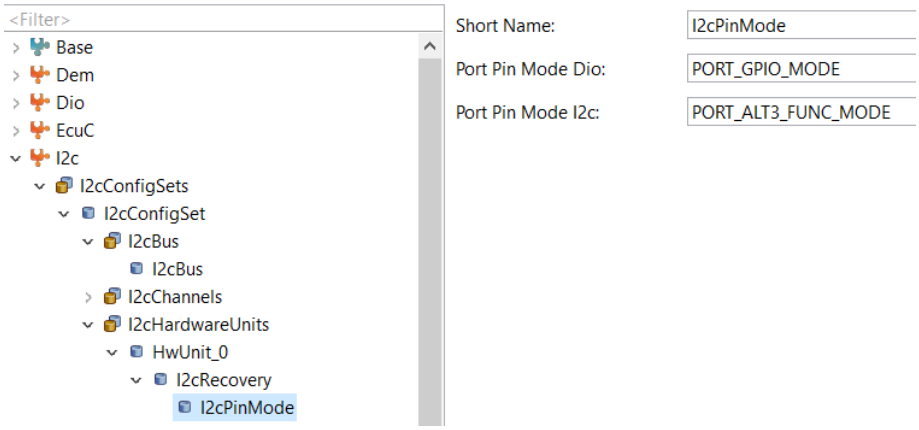


Figure 5-5 Configuration of the port driver interface

The parameters which are passed to the port driver to switch the port to the corresponding mode, can be configured in container I2cPinMode for the specific hardware unit. With the parameter “Port Pin Mode Dio” the mode is set to GPIO and with the parameter “Port Pin Mode I2c” the mode is set to I2C.

5.1.5 Base address

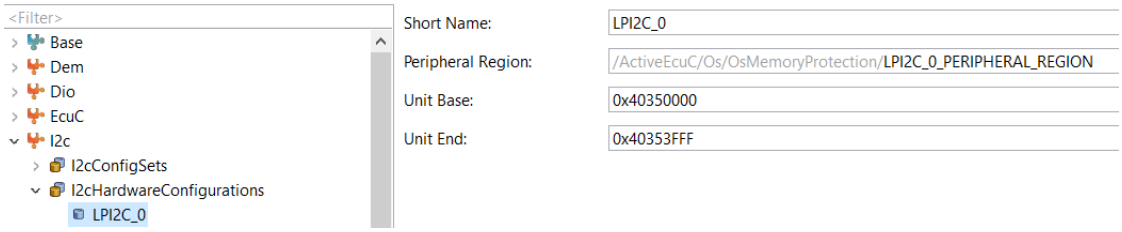


Figure 5-6 Base address of the peripheral units

Caution

Be sure to configure a valid base address if adding a hardware configuration. A misconfiguration of this parameter leads to a CPU crash or can cause an undefined behavior. Also verify the selected hardware configuration fits to the chip derivative.

6 Glossary and Abbreviations

6.1 Glossary

Term	Description
DaVinci Configurator Pro	Configuration and generation tool for MICROSAR components
PORT	A PORT represents a whole configurable port on this microcontroller (typically controlled by one hardware register set).
I2C	Inter Integrated Circuit, also known as IIC or I2C
DIO	The DIO provides services for reading and writing to/from DIO ports

Table 6-1 Glossary

6.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
ECU	Electronic Control Unit
GPIO	General Purpose Input Output
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
OS	Operating System

Table 6-2 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com